# Do LLMs Really Compose Skills? A Dichotomy in Compositional Ability

COLM 2024, University of Wisconsin-Madison: when scaling helps LLMs combine two learned tasks, and when it doesn't

We have come to expect a particular trick from large language models: show them a few examples in the prompt, and they will pick up a new task on the fly. That is in-context learning. But real reasoning rarely means repeating a single operation. More often it means chaining together several skills you already have.

That raises a natural question that has gone surprisingly untested: if a model has separately learned task A and task B, can it automatically combine them when it meets a new, unseen task that needs both at once? For a human this is almost reflexive. For an LLM, the answer turns out to be far less obvious.

This COLM 2024 paper answers with a sharp dichotomy. When the two sub-tasks act on separate parts of the input, models compose them well and get steadily better with scale. But when a task demands genuine multi-step, chained reasoning, models tend to fail outright, and making the model bigger does not rescue them.

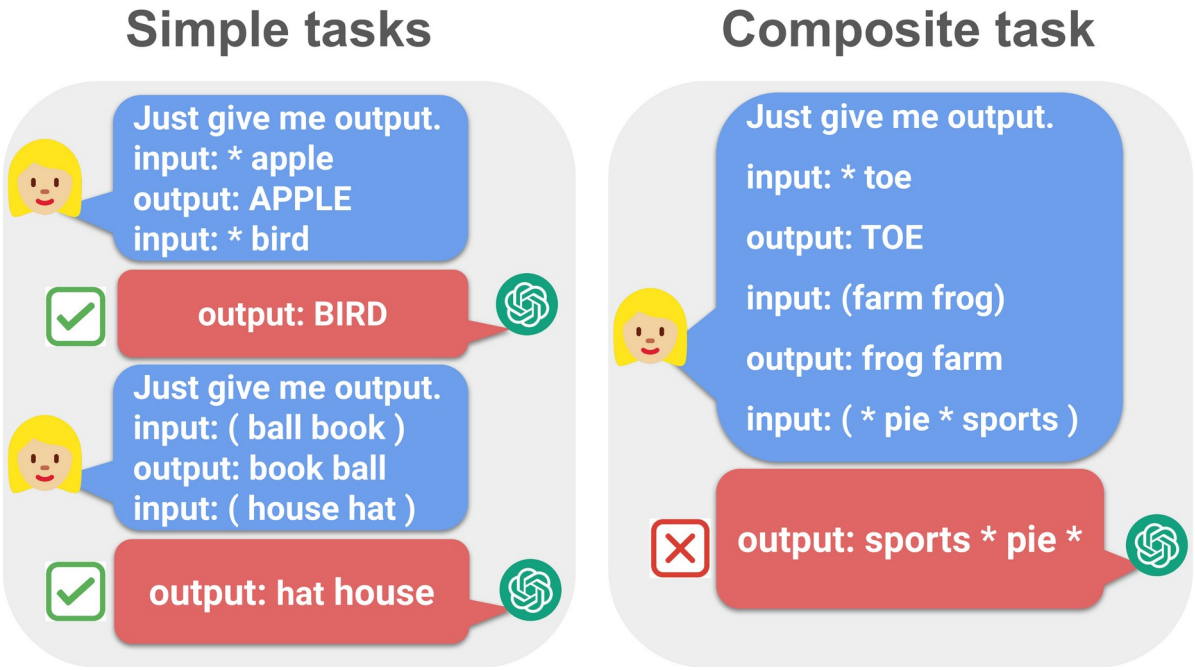


Figure 1. Compositional in-context learning, illustrated. Given simple-task demonstrations (words after * are capitalized; words in parentheses are swapped), the model solves each simple task on its own but fails to compose them on the composite task, producing a wrong output.

Paper: https://arxiv.org/abs/2407.15720 Code: https://github.com/OliverXUZY/LLM_Compose

## A task humans take for granted, and models often can't do

Start with a concrete example. Give the model two sets of simple demonstrations: words marked with * should be capitalized, and the two words inside parentheses should be swapped. Tested one at a time, the model handles both cleanly. But feed it a single input containing both a starred word and a parenthesized pair, and ask for both operations at once, and even frontier models like GPT-4 and Claude 3 routinely stumble.

The point is that each individual skill is already in hand; the only thing that breaks is the act of composing them. That tells us the failure is not about raw capability but about composition itself. The paper builds a controlled study around exactly this gap, asking whether LLMs genuinely have compositional ability, or whether that ability is far more limited than their per-skill competence would suggest.

## Measuring compositional ability: four settings

The authors measure under standard in-context learning: each prompt carries K=10 demonstrations plus one test input. For every composite task they define four controlled settings, designed to cleanly separate whether a model can do the individual skills from whether it can combine them.

The first two settings test each simple sub-task on its own. The third is the composite setting: the demonstrations are drawn only from the simple sub-tasks, but the test input requires both. The fourth, composite-in-context, swaps in composite demonstrations as well, giving a gold-standard upper bound for how well the model could do if it were shown the composition directly. Together, the four make it possible to tell whether what's missing is capability or composition.

## The dichotomy: separable tasks vs. compose-by-step tasks

The paper's central finding is that composite tasks are not all alike; they split into two very different families. Separable tasks apply their two operations to different parts of the input, so the operations don't interfere, for example capitalizing some words while adding to a separate number. Compose-by-step tasks require chaining the two operations into a single multi-step reasoning path over shared input, for example capitalize, then swap. Models compose the former and improve with scale; on the latter they collapse.

The figure below shows a textbook compose-by-step case: Capitalization & Swap. The x-axis is model scale, the y-axis is exact-match accuracy, and the four lines correspond to the four settings above. Watch where the red composite curve sits relative to the other three.

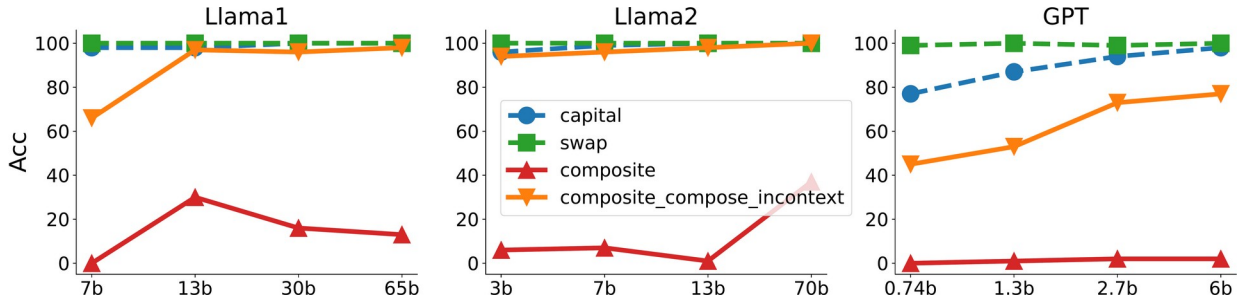


Figure 2. Exact-match accuracy (y-axis) vs. model scale (x-axis, "b" = billion parameters) on the Capitalization & Swap task. capital and swap are the two simple tasks; composite uses simple-task demonstrations with a composite test input; composite_compose_incontext uses composite demonstrations and test input, the gold-standard upper bound.

The result is stark. The capital and swap curves hug 100%, so the individual skills are fully in hand, and the composite-in-context upper bound (orange) is high too, so the model clearly could do the composition if shown it directly. Yet the actual composite curve (red) lingers around 20% or below, and it barely moves as the model grows from 7b to 65b or 70b. On compose-by-step tasks, scale simply does not buy improvement.

Switch task families and the story flips. The next figure shows translation composite tasks, now scored by word error rate (WER, lower is better), covering Phrase Recombination with Longer Chain and Passive-to-Active with Object-to-Subject transformation. These composites sit closer to the separable end of the spectrum.

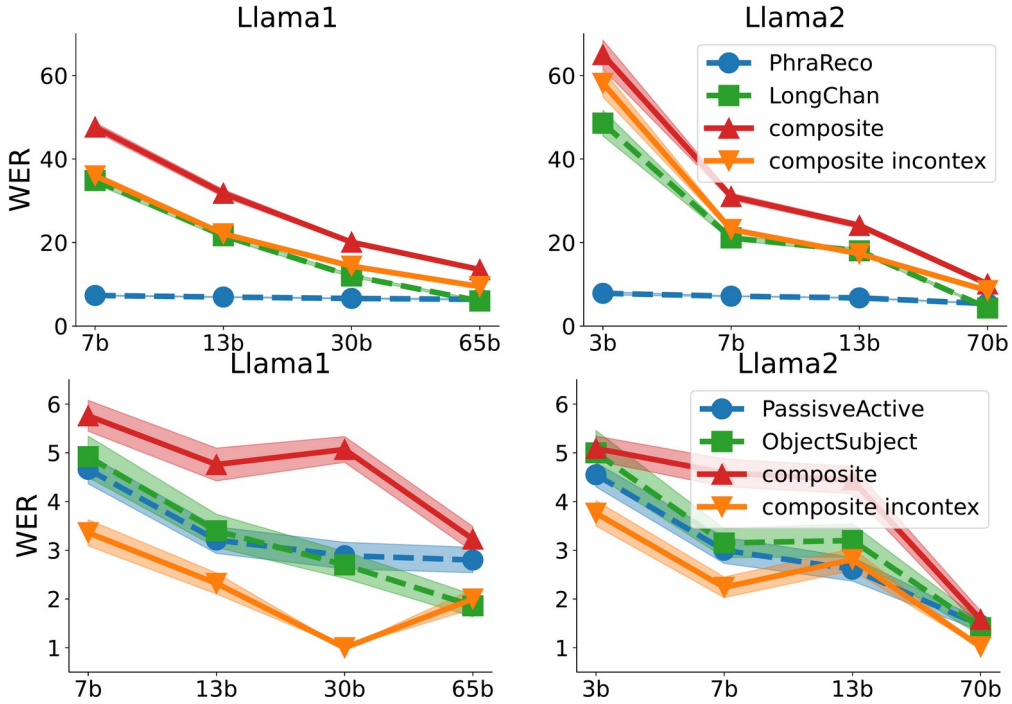


Figure 3. Word error rate (WER) vs. model scale on composite linguistic translation tasks. Dashed lines are simple tasks, solid lines are composite tasks. Rows: (T1) Phrase Recombination with Longer Chain; (T2) Passive-to-Active and Object-to-Subject transformation. Columns: different models. Lines: different evaluation settings.

Here the composite curve (solid red) sees its WER fall steadily as the model grows, closing in on both the composite-in-context bound (orange) and the simple tasks. This is the other half of the dichotomy: when the sub-tasks are separated enough in the input, scaling really does lift compositional performance. Whether it's worth spending more compute on a composite task can, to a large extent, be predicted from the task's structure alone.

## Why does this happen? A linear self-attention account

The paper does not stop at the phenomenon; it offers a theoretical explanation. The authors analyze a one-layer, single-head linear self-attention network and prove that composition succeeds precisely when the two simple tasks have confined support, meaning each occupies its own separate feature subspace of the input embedding.

When the subspaces are separate, as in separable tasks, the model can superpose the two operations without interference and composition holds, and it improves as the underlying simple tasks improve with scale. When the subspaces overlap, as in compose-by-step tasks, the two operations become entangled in the same representation and composition breaks down, no matter how many parameters are added. The theory thus lines up exactly with the empirical dichotomy and the scaling behavior observed in the experiments.

## The numbers at a glance

**Table 1. Representative results across settings (Llama / GPT families, K=10 in-context demonstrations).**

| Setting | Result |
|---|---|
| Simple tasks (capitalization / swap), Llama | ~90% |
| Compose-by-step (Capitalization & Swap) | ≤ 20%, flat with scale |
| Capitalization + Two Sum (larger scale) | 44% |
| Swap + Two Sum (larger scale) | 66% |
| GPT-4 / Claude 3 on the composite task | also fail |

Evaluated across Llama-1 (7b–65b), Llama-2 (3b–70b), Llama-3, and GPT (0.74b–6b) scales.

## Takeaway and boundaries

The conclusion is both clean and useful: LLMs can reliably combine two skills only when those skills act on separate parts of the input. Once a task genuinely requires chained, multi-step reasoning, models fail at the composition step, and simply adding parameters will not bring the ability back.

The boundary is worth stressing. When the model is shown composite demonstrations directly, accuracy recovers immediately, which means the model is not short on representational power, it is short on the ability to assemble the composition from simple-task demonstrations alone. So whether a composite reasoning task will benefit from more compute is largely dictated by its structure: separable, confined-support tasks scale, while deeply entangled multi-step tasks do not. That gives a practical rule of thumb for which composite tasks an LLM can be trusted to solve.